DataModel

0. The three subgraphs

Everything lives in **one Neo4j database**, but in three label-namespaced subgraphs that never share labels. This is deliberate: it keeps the ontology queryable with the same driver/session as the data, avoids a second datastore, and lets validation (AC-7) run without touching domain data.

#	Subgraph	Labels	Purpose	Spec ref
1	**Ontology (meta)**	`:EntityType`, `:AttributeDef`, `:RelationshipType`	Drives type-ahead, filter panel, attribute selector, relationship picker, and server-side validation — *without a live data query*	§5.1, §6.1, §6.2, §6.4
2	**Domain graph**	`:Entity` + one label per concrete type (`:Personnel`, `:TechnologySystem`, …)	The actual entity graph being explored	§5.1
3	**Exploration store**	`:Exploration`, `:Column`, `:Filter`, `:Report`	Saved explorations + generated report snapshots	§5.2, §6.6, FR-9

> **Why not a relational store for #3?** FR-9 needs a *tree* with arbitrary depth and branching (§3 Step 3, depth 5–20). In SQL that's an adjacency table plus recursive CTEs; in Neo4j it's the native shape. One database, one driver, one transaction manager. Trade-off: you give up the relational store's easy row-level auditing — noted in §9 below.

---

1. Ontology subgraph (the schema/metadata store)

This is the answer to Assumption #1: **build it explicitly, don't derive it from the graph.** Deriving it (`CALL db.schema.visualization()` / `apoc.meta.schema`) gives you what *exists*, not what's *allowed*, and it can't give you labels, hierarchy, or `filterable` flags. Seed it from config (§`cypher/01_ontology.cypher`); optionally reconcile against `apoc.meta.schema` in a nightly job to catch drift.

Nodes

```
(:EntityType {
   id: 'technology_system',      // stable, snake_case — the entityTypeId in
every API
   label: 'Technology System',   // display
   nodeLabel: 'TechnologySystem',// the Neo4j label used in the domain
subgraph
   abstract: false,              // 'system' is abstract: no instances carry
it alone
   searchText: 'technology system tech sys' // lowercased, for type-ahead
})

(:AttributeDef {
   id: 'technology_system.location',   // composite, unique
   name: 'location',                   // property key on the domain node
   label: 'Location',
   dataType: 'STRING',                 // STRING | NUMBER | DATE | BOOLEAN |
ENUM
   filterable: true,
   selectable: true,                   // may appear in a report
   enumValues: []                      // only for ENUM
})

(:RelationshipType {
   id: 'OWNS',                  // == the Neo4j relationship type in the
domain graph
   label: 'owns',
   inverseLabel: 'owned by',    // shown when the user picks direction IN
   relType: 'OWNS'
})
```

Relationships

```
(:EntityType)-[:CHILD_OF]->(:EntityType)        // hierarchy: TechnologySystem
-CHILD_OF-> System
(:EntityType)-[:HAS_ATTRIBUTE]->(:AttributeDef)
(:RelationshipType)-[:FROM_TYPE]->(:EntityType)
(:RelationshipType)-[:TO_TYPE]->(:EntityType)
```

Two modelling decisions worth defending

(a) Attribute inheritance is by traversal, not duplication. `TechnologySystem` does *not* copy `System`'s attributes. §6.2 resolves the effective attribute set with:

```
MATCH (t:EntityType {id: $entityTypeId})-[:CHILD_OF*0..]->
(ancestor:EntityType)
MATCH (ancestor)-[:HAS_ATTRIBUTE]->(a:AttributeDef)
RETURN DISTINCT a
```

One place to add an attribute; every subtype gets it. Same for relationships: `Personnel -OWNS-> System` is automatically offerable on a `Personnel` node and *targets* `System`, which resolves to all its subtypes.

(b) Type hierarchy in the domain graph is expressed with multiple labels, not a property. A technology system node carries `:Entity:System:TechnologySystem`. So "find all Systems" is `MATCH (n:System)` — an index-backed label scan — instead of `WHERE n.entityTypeId IN [list of subtype ids]`, which is a property filter and roughly an order of magnitude worse at 10k+ nodes. `EntityType.nodeLabel` is what the query builder injects; `EntityType.id` is what the API speaks. Never let a client-supplied string reach a label position — see §5.

---

2. Domain subgraph

```
(:Entity:Personnel {
   id: 'P-123',                 // stable business id, unique across :Entity
   entityTypeId: 'personnel',   // denormalized for cheap DTO mapping + audit
   firstName: 'Jane', lastName: 'Doe', title: 'VP Engineering',
   division: 'Y',
   displayLabel: 'Jane Doe — VP Engineering',  // powers §6.3 instance search
   searchText: 'jane doe vp engineering'       // lowercased, full-text
indexed
})

(:Entity:System:TechnologySystem {
   id: 'S-9', entityTypeId: 'technology_system',
   name: 'Billing Platform', location: 'us-east', ...
})

(:Entity:Vendor { id: 'V-3', entityTypeId: 'vendor', name: 'Acme',
contactEmail: '...' })
```

Edges are directed and typed exactly as `RelationshipType.relType`:

```
(:Personnel)-[:OWNS]->(:System)
(:Personnel)-[:USES]->(:System)
(:Personnel)-[:REPORTS_TO]->(:Personnel)
```

```
(:Personnel)-[:HAS_IMMEDIATE_REPORTEE]->(:Personnel)
(:System)-[:MAINTAINED_BY]->(:Vendor)
```

Direction (§5.1). The spec asks for direction to be explicit. Two options:

- **Store both `REPORTS_TO` and `HAS_IMMEDIATE_REPORTEE` as separate edges** (what the spec's §6.4 example implies), or
- **Store one edge and let the picker offer it in both directions** (`direction: IN` renders `inverseLabel`).

Recommendation: store one edge (`REPORTS_TO`), offer both directions. A stored inverse is a denormalization you must keep consistent on every write, and there's no write path in this phase to keep it consistent *with*. The exploration model already carries `"direction": "OUT" | "IN"` , so the query builder emits `-[:REPORTS_TO]->` or `<-[:REPORTS_TO]-` from the same `RelationshipType` . §6.4 then returns two entries for one stored type — this is the one place the ontology response is richer than the ontology graph. If the product owner insists on genuinely distinct semantics (reportee ≠ inverse of reports-to, e.g. dotted-line reports), model them as two `RelationshipType` s and two edge types; the model supports both. **Flag this one — it's the highest-leverage open question in the data model.**

Constraints & indexes (see `cypher/00_constraints.cypher`)

Object	Why
`CONSTRAINT ON (n:Entity) ASSERT n.id IS UNIQUE`	stable node id; also creates the lookup index used by every filter that pins an instance
`CONSTRAINT ON (t:EntityType) ASSERT t.id IS UNIQUE`	ontology integrity
`CONSTRAINT ON (a:AttributeDef) ASSERT a.id IS UNIQUE`	composite `type.attr` id
`CONSTRAINT ON (r:RelationshipType) ASSERT r.id IS UNIQUE`	
`CONSTRAINT ON (e:Exploration) ASSERT e.id IS UNIQUE`	
`CONSTRAINT ON (c:Column) ASSERT c.uid IS UNIQUE`	global uid; `columnId` ('c1') is only unique *within* an exploration
Full-text `entityTypeSearch` on `:EntityType(label, searchText)`	NFR-1: <300ms type-ahead over ~10k types
Full-text `entitySearch` on `:Entity(displayLabel, searchText)`	§6.3 instance search
Range index per filterable attribute, per type (`:Personnel(division)` ,	FR-2 filters; generate these *from the ontology* — see

Object	Why
`:TechnologySystem(location)`, …)	`cypher/02_domain_indexes.cypher`

The per-attribute index list is derived, not hand-maintained: every `AttributeDef {filterable: true}` gets an index on its owning type's `nodeLabel`. A startup task can emit and run these.

---

3. Exploration subgraph (§5.2, FR-9)

The spec's JSON tree maps to the graph almost one-to-one.

```
(:Exploration {
   id: 'exp-001', name: 'System ownership review',
   ownerId: 'u-42', createdAt: datetime(), updatedAt: datetime(),
   version: 3                        // optimistic locking for PUT
/explorations/{id}
})

(:Column {
   uid: 'exp-001:c1',                // globally unique
   columnId: 'c1',                   // as it appears in the API tree + report
rows
   entityTypeId: 'personnel',
   selectedAttributes: ['firstName','lastName','title'],  // list property,
order preserved
   position: 0                       // sibling ordering, so reload renders
identically
})

(:Filter {
   uid: 'exp-001:c1:f1',
   attribute: 'division',
   op: 'EQUALS',                     // EQUALS | CONTAINS | IN | GT | GTE | LT
| LTE | BETWEEN | IS_NULL
   value: 'Y',                       // scalar
   values: []                        // for IN / BETWEEN
})
```

```
(:Exploration)-[:ROOT_COLUMN]->(:Column)
(:Column)-[:HAS_FILTER]->(:Filter)
(:Column)-[:NEXT { relTypeId: 'OWNS', direction: 'OUT', position: 0 }]->
(:Column)
```

`[:NEXT]` **is the spec's** `children[].relationship` — the edge type + direction ride on the edge, which is exactly where they belong. Reading an exploration back is one query; there is no recursive CTE.

Three things this buys you:

- **FR-6 (remove a column removes its descendants)** is `MATCH (c)-[:NEXT*0..]->(d) DETACH DELETE d` — the cascade is structural, not application logic you can forget.
- **AC-9 (save → reload → generate reproduces the same report)** holds because `selectedAttributes` order and `position` are persisted.
- **Reuse/analytics later**: "which entity types do people actually explore from?" is a query over `:Column`, free.

Report snapshots (optional, but §6.7 returns a `reportId` — something must own it):

```
(:Report { id:'rep-777', generatedAt: datetime(), truncated: false, rowCount:
412,
           payload: '<json>' })     // store the materialized rows as a JSON
string, or in blob storage if large
(:Report)-[:GENERATED_FROM]->(:Exploration)
```

Keep `payload` out of Neo4j if reports get big — Neo4j is not a blob store. Cap it (say 1 MB) and spill to S3/filesystem beyond that, keeping only the pointer.

---

4. From exploration tree → Cypher (§6.7, the recursive query builder)

The builder walks the exploration tree depth-first and emits **one** `MATCH` **for the root, then one** `OPTIONAL MATCH` **per child branch**, with every user value bound as a parameter (AC-10).

Given the spec's §5.2 example, the generated query is:

```
// root column c1
MATCH (c1:Personnel)
WHERE c1.division = $c1_f0
// branch c1 -OWNS-> c2 -MAINTAINED_BY-> c3
OPTIONAL MATCH (c1)-[:OWNS]->(c2:System)
OPTIONAL MATCH (c2)-[:MAINTAINED_BY]->(c3:Vendor)
// branch c1 -HAS_IMMEDIATE_REPORTEE-> c4
OPTIONAL MATCH (c1)-[:HAS_IMMEDIATE_REPORTEE]->(c4:Personnel)
RETURN
```

```
  { id: c1.id, firstName: c1.firstName, lastName: c1.lastName, title: c1.title
} AS c1,
  CASE WHEN c2 IS NULL THEN NULL ELSE { id: c2.id, name: c2.name, location:
c2.location } END AS c2,
  CASE WHEN c3 IS NULL THEN NULL ELSE { id: c3.id, name: c3.name,
contactEmail: c3.contactEmail } END AS c3,
  CASE WHEN c4 IS NULL THEN NULL ELSE { id: c4.id, firstName: c4.firstName,
lastName: c4.lastName, title: c4.title } END AS c4
SKIP $skip LIMIT $limit
```

Notes that matter:

- `OPTIONAL MATCH` **, not** `MATCH` **, for children.** With `MATCH`, a Personnel who owns nothing vanishes from the report entirely. The example narrative ("X has direct reportees P2, P3, P4") expects the root to survive an empty branch. Cross-product across sibling branches is intentional and is exactly what a flat row-based report (§6.7 `rows`) means — but it's also how you get 12 × 30 × 8 rows. Hence NFR-2's cap.
- `LIMIT $limit + 1` in practice: fetch cap+1 rows, and if you got cap+1, return cap and set `truncated: true` (AC-8).
- **Labels and relationship types are injected from the ontology, never from the request.** The request carries `entityTypeId: 'personnel'`; the builder looks up `EntityType.nodeLabel = 'Personnel'` and injects *that*. An unknown id fails validation with a 400 before any string is built (AC-7). Attribute names get the same treatment — validated against `AttributeDef`, then injected. Values are *always* `$params`. That's the full injection story.
- **Cycle guard (Assumption #7).** Because every hop is an explicit fixed-length pattern (no `*` variable-length), the traversal cannot loop — depth is bounded by the tree the user drew (≤20). If you later add variable-length hops ("all reports transitively"), you *must* add `apoc.path.expandConfig` with `uniqueness: 'NODE_GLOBAL'`. Recommend keeping v1 fixed-hop only and flagging transitive traversal as a v2 feature.
- **Preview counts (§6.5, FR-7)** reuse the same builder, swapping the `RETURN` for `count(DISTINCT c1) AS c1, count(DISTINCT c2) AS c2, …` and dropping SKIP/LIMIT.

Full worked builder in `java/ExplorationCypherBuilder.java`; all endpoint queries in `cypher/04_api_queries.cypher`.

---

5. Validation gate (AC-7, AC-10)

Before any query is built, `ExplorationValidator` runs the exploration tree against the ontology in **one** Cypher round-trip (`cypher/05_validate_exploration.cypher`) and collects *all* errors,

not the first:

1. Every `entityTypeId` exists and is not `abstract` — *or* is abstract and therefore resolves to a label matching all subtypes (allowed; `System` is a legal column type).
2. Every `[:NEXT].relTypeId` exists, and its `FROM_TYPE` / `TO_TYPE` (respecting `direction`) are compatible with the parent and child column types, **accounting for hierarchy** (a relationship declared to `System` is valid into a `TechnologySystem` column).
3. Every filter attribute exists on the column's type (or an ancestor) and has `filterable: true`, and its `op` is legal for its `dataType` (AC-2: `CONTAINS` on NUMBER → 400).
4. Every selected attribute exists and has `selectable: true`.
5. Tree depth ≤ `maxDepth` (config, default 20), total columns ≤ `maxColumns` (default 50).

Only then does the builder run. **No DB call to the domain graph on an invalid tree** — the validation query only touches the ontology subgraph, which satisfies the letter and spirit of AC-7.

---

6. Open questions this model forces you to answer

1. **Inverse relationships** (§2 above) — one edge with `direction`, or two stored edge types? Highest-impact question. *My recommendation: one edge.*
2. **Abstract types as columns** — can a user pick `System` (getting both subtypes mixed into one column), or must they pick a leaf? The model allows both; the UI should probably allow it, but the report's attribute list then has to be the *intersection* of subtype attributes. Confirm.
3. **Report payload storage** — in-graph JSON vs object storage, and retention. Reports are materialized snapshots; they will accumulate.
4. **Row-level security** (Assumption #6) — if the graph must enforce per-user visibility, the cleanest hook is a mandatory `WHERE $userId IN c1.visibleTo` predicate injected by the builder on every column, backed by a list property or a `[:VISIBLE_TO]` edge. That decision changes the domain model, so decide before M2.
5. **Relationship picker mode** (Assumption #2) — the ontology supports schema-driven for free. Data-driven ("only relationships this node actually has") is `MATCH (n:Entity {id:$id})-[r]->() RETURN DISTINCT type(r)` — cheap for one pinned node, expensive for a whole filtered column. *Recommend: schema-driven picker, data-driven only when a single instance is pinned.*